PATTERN-DRAWING ROBOT USING MATLAB & COPPELIASIM

A MICRO-PROJECT REPORT

submitted by

Adil Naz Muhammed
(P250285)

to
the APJ Abdul Kalam Technological University
in partial fulfillment of the requirements for the award of the Degree
of
Master of Technology
in
Robotics and Automationfor
221TIA003-KINEMATICS, DYNAMICS AND CONTROL OF ROBOTS

Department of Electronics & Communication Engineering
College of Engineering Trivandrum
Thiruvananthapuram
695016
November 2025

DECLARATION

I, Adil Naz Muhammed, hereby declare that the project report **Pattern-Drawing Robot Using MATLAB & CoppeliaSim** submitted for partial fulfillment of the requirements for the course 221TIA003-KINEMATICS, DYNAMICS AND CONTROL OF ROBOTS is a bonafide work done by me under supervision of Prof. Merlin Mon Mathew. This submission represents my ideas in my own words and where ideas or words of others have been included, I have adequately and accurately cited and referenced the original sources. I also declare that I have adhered to ethics of academic honesty and integrity and have not misrepresented or fabricated any data or idea or fact or source in my submission. I understand that any violation of the above will be a cause for disciplinary action by the institute and/or the University and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been obtained. This report has not been previously formed the basis for the award of any degree, diploma or similar title of any other University.

Place	:	Trivandrum	Signature of student	:	
Date	:	November 26, 2025	Name of student	:	Adil Naz Muhammed

DEPARTMENT OF ELECTRONICS & COMMUNICATION ENGINEERING ELECTRONICS & COMMUNICATION ENGINEERING

College of Engineering Trivandrum

Thiruvananthapuram

695016

CERTIFICATE

This is to certify that the report entitled **Pattern-Drawing Robot Using MATLAB & CoppeliaSim** submitted by **Adil Naz Muhammed** to the APJ Abdul Kalam Technological University in partial fulfillment of the requirements for the award of the Degree of Master of Technology in Robotics and Automation in is a bonafide record of the project work carried out by him/her under my/our guidance and supervision. This report in any form has not been submitted to any other University or Institute for any purpose.

Project Guide

Name : Merlin Mon Mathew

Signature : .....

ACKNOWLEDGMENT

I wish to record my indebtedness and thankfulness to all who helped me prepare this Project Report titled **Pattern-Drawing Robot Using MATLAB & CoppeliaSim** and present it satisfactorily. I am especially thankful for my guide and supervisor Prof. Merlin Mon Mathew in the Electronics & Communication Engineering Department for giving me valuable suggestions and critical inputs in the preparation of this report. Last but not least for God, my parents and Generative AI tools which kept me going, keeping up the grind.

Adil Naz Muhammed
(Reg. No. P250285)
M. Tech. (Robotics and Automation)
Department of Electronics & Communication Engineering
College of Engineering Trivandrum

ABSTRACT

This project presents the design and simulation of a 3-DOF Cartesian pattern-drawing robot using MATLAB and CoppeliaSim. The robot traces geometric shapes such as circles and squares by following trajectories generated in MATLAB and executed in CoppeliaSim through the ZeroMQ Remote API. Initial control development explored PID-based joint control, but instability resulted from the non-motorized prismatic joint model within CoppeliaSim 4.6. A smooth incremental kinematic controller was developed as the final solution, providing stable and accurate motion suitable for Cartesian gantry systems. The report documents the modelling, control design, debugging process, performance analysis, and an ideal real-world hardware implementation.

CONTENTS

ACKNOWLEDGMENT **i**

ABSTRACT **ii**

Chapter 1. INTRODUCTION TO THE PROJECT **1**

Chapter 2. SYSTEM ARCHITECTURE **3**

2.1 Software Framework 3

2.2 Robot Model in CoppeliaSim 4

2.3 MATLAB–CoppeliaSim Communication 4

2.4 Real-Time Execution Loop 6

Chapter 3. ROBOT DESIGN AND KINEMATICS **7**

3.1 Mechanical Design Concept 7

3.2 Direct Kinematics 7

3.3 Advantages of Cartesian Kinematics 8

3.4 Simulation Constraints and Considerations 8

Chapter 4. CONTROL APPROACHES **10**

4.1 Overview 10

4.2 Initial PID Control Strategy 10

4.3 Analysis of PID Instability 11

4.4 Transition to Kinematic Incremental Control 12

4.5 Advantages of Incremental Kinematic Control 12

4.6 Final Algorithm 13

Chapter 5. TRAJECTORY GENERATION **14**

5.1 Overview 14

5.2 Square Trajectory 14
5.3 Circular Trajectory 15
5.4 MATLAB Implementation 16
5.5 Trajectory Resolution 16
5.6 Extensibility 17

Chapter 6. MATLAB–COPPELIASIM INTEGRATION AND RESULTS 18
6.1 Integration via ZeroMQ Remote API 18
6.2 Incremental Kinematic Controller 19
6.3 Experimental Setup 20
6.4 Results and Analysis 20
6.5 Summary 21

Chapter 7. IDEAL HARDWARE IMPLEMENTATION 22
7.1 Overview 22
7.2 Mechanical Structure 22
7.3 Actuation: Stepper Motors and Drivers 23
7.4 Control Electronics 24
7.5 Firmware Architecture 24
7.6 Mapping Simulation to Hardware 25
7.7 Potential Hardware Enhancements 25
7.8 Summary 25

REFERENCES 26

Chapter 1

INTRODUCTION TO THE PROJECT

Robotic drawing systems, also known as plotters or Cartesian gantry robots, are widely used in manufacturing, laboratory automation, rapid prototyping, and educational robotics. These systems rely on precise end-effector motion to reproduce predefined patterns such as lines, curves, and complex shapes. With the increasing availability of simulation tools and high-level control environments, there is a growing interest in developing flexible, simulation-driven robotic drawing platforms that can be extended into real hardware. This project focuses on the design, simulation, and control of a pattern-drawing robot using MATLAB and CoppeliaSim. The simulated robot is configured as a three-degree-of-freedom (3-DOF) Cartesian mechanism consisting of prismatic joints that independently control motion along the X, Y, and Z axes. This configuration offers several advantages: simple mechanical construction, intuitive control, and direct mapping between joint displacement and end-effector position.

A key objective of the project is to develop a trajectory-tracking controller capable of executing geometric patterns smoothly and accurately within the simulation environment. Initial experiments involved implementing a classical PID-based control strategy. Although PID control is widely used in motorized robotic systems, it proved unsuitable for this specific simulation. CoppeliaSim version 4.6 models prismatic joints as kinematic or passive dynamic elements without internal motor actuation or velocity-level control. As a result, torque- or velocity-based PID controllers produced unstable behaviour, oscillations, and abrupt motion. To achieve stable operation, the control approach was revised. A smooth incremental kinematic

controller was developed in MATLAB, where the end-effector is advanced toward the desired trajectory in small linear steps. This method mirrors the real-world control philosophy of CNC machines and 3D printers, where motion is governed by incremental feedrate commands rather than torque-level PID control. The final controller successfully executes circle and square trajectories with high smoothness and predictable motion.

Beyond simulation, the project also outlines how such a system would be implemented in hardware using stepper motors, belt-drive mechanisms, micro-controller firmware, and G-code-style motion commands. The simulation-driven design approach adopted here provides a strong foundation for real-world prototyping and control development. This report documents the modeling, control design, debugging process, results, and ideal hardware implementation of the MATLAB–CoppeliaSim pattern-drawing robot.

Chapter 2

SYSTEM ARCHITECTURE

This chapter describes the overall architecture of the pattern-drawing robot, including the software framework, communication interface, and the structure of the 3-DOF Cartesian manipulator simulated in CoppeliaSim. The architecture was designed to enable MATLAB-driven, real-time trajectory execution with accurate and predictable end-effector motion.

2.1 SOFTWARE FRAMEWORK

The system consists of three major software components:

- **MATLAB Controller:** Responsible for trajectory generation, control logic, incremental motion updates, and communication with the simulator.
- **ZeroMQ Remote API:** A lightweight, message-based communication interface that allows MATLAB to control and query CoppeliaSim objects in real time.
- **CoppeliaSim Simulation Environment:** Provides a physics-based and kinematic environment for simulating robotic mechanisms, enabling joint manipulation, scene visualization, and debugging.

Figure 2.1 illustrates the high-level data flow between the components.

Figure 2.1: System architecture showing MATLAB, ZeroMQ Remote API, and CoppeliaSim.

2.2 ROBOT MODEL IN COPPELIASIM

The robot is modeled as a 3-DOF Cartesian manipulator composed of three prismatic joints arranged orthogonally, enabling motion along the X, Y, and Z axes. The prismatic joints directly correspond to linear displacements, making the robot particularly suitable for planar motion tasks such as pattern drawing.

Joint Modes in CoppeliaSim

In CoppeliaSim version 4.6, prismatic joints support three primary modes:

- **Kinematic Mode** – The joint position is directly set by the user or controller, with no physics or actuation dynamics.
- **Dynamic Mode** – The joint responds to forces but does not include an integrated motor or velocity controller for prismatic joints.
- **Dependent Mode** – The joint follows another joint with a linear dependency.

Due to the lack of motor actuation for prismatic joints, torque-based PID control is not applicable. This motivates the use of a feedrate-limited, incremental kinematic control strategy.

2.3 MATLAB–COPPELIASIM COMMUNICATION

The ZeroMQ Remote API allows MATLAB to:

- Acquire joint object handles.

- Read current joint positions at runtime.
- Send position-update commands to prismatic joints.
- Execute and stop simulation programmatically.

The following MATLAB pseudo-code demonstrates the basic communication loop:

```
client = RemoteAPIClient();
sim = client.getObject('sim');

hX = sim.getObject('/Prismatic_joint_x');
hY = sim.getObject('/Prismatic_joint_y');
hZ = sim.getObject('/Prismatic_joint_z');

sim.startSimulation();

while running
    q = [ sim.getJointPosition(hX), ...
          sim.getJointPosition(hY), ...
          sim.getJointPosition(hZ) ];

    q_new = q + small_increment;

    sim.setJointPosition(hX, q_new(1));
    sim.setJointPosition(hY, q_new(2));
    sim.setJointPosition(hZ, q_new(3));
end
```

```
sim.stopSimulation();
```

2.4 REAL-TIME EXECUTION LOOP

The real-time control loop operates at a fixed timestep, typically $dt = 0.02$ seconds. At each iteration:

1. MATLAB computes the next desired Cartesian point along the trajectory.
2. The displacement between the current position and target point is clipped to a maximum step size to maintain smooth motion.
3. Updated joint positions are sent to CoppeliaSim using `sim.setJointPosition`.
4. The simulator updates the robot configuration and returns joint readings.

This architecture ensures stable, smooth, and deterministic motion suitable for pattern generation tasks.

Chapter 3

ROBOT DESIGN AND KINEMATICS

3.1 MECHANICAL DESIGN CONCEPT

The simulated robot is modeled as a three-degree-of-freedom (3-DOF) Cartesian manipulator. Each axis corresponds to a prismatic joint aligned with the global X, Y, and Z directions. This architecture is widely used in gantry-style machines such as CNC plotters, laser engravers, and 3D printers due to its mechanical simplicity and high positional accuracy.

Prismatic Joint Arrangement

The robot consists of:

- **Joint X**: Horizontal motion along the X-axis.
- **Joint Y**: Horizontal motion along the Y-axis.
- **Joint Z**: Vertical motion used primarily for pen engagement.

Each joint directly produces linear displacement in its corresponding axis. This eliminates the need for inverse kinematics, reduces computational load, and makes the system highly suitable for pattern drawing tasks where absolute Cartesian positioning is required.

3.2 DIRECT KINEMATICS

The direct kinematics (DK) of the robot is given by the mapping:

$$\mathbf{p}_{ee} = \begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} q_x \\ q_y \\ q_z \end{bmatrix},$$

where:

- q_x, q_y, q_z are the prismatic joint displacements,
- $\mathbf{p}_{ee}$ is the end-effector (pen) position in 3D space.

Since the robot is purely Cartesian, there is no rotational coupling or nonlinear mapping. The system exhibits a one-to-one correspondence between joint space and Cartesian space, significantly simplifying controller design and trajectory execution.

3.3 ADVANTAGES OF CARTESIAN KINEMATICS

The choice of a 3-axis Cartesian robot provides several benefits:

- **Linearity**: Joint displacements map directly to Cartesian coordinates.
- **Decoupled Motion**: Each axis can be controlled independently.
- **High Precision**: Ideal for drawing continuous curves and geometric shapes.
- **Ease of Control**: No Jacobians or inverse kinematics are required.

For tasks such as generating circles, squares, and other 2D patterns, these properties lead to excellent controllability and predictable motion behavior.

3.4 SIMULATION CONSTRAINTS AND CONSIDERATIONS

In CoppeliaSim version 4.6+, prismatic joints are implemented as kinematic or passive dynamic elements without built-in motor actuation or internal PID con-

trollers. As a result:

- Torque-level or velocity-level control cannot be applied.
- PIDs controlling joint dynamics do not behave physically.
- Motion must be expressed through direct Cartesian or kinematic updates.

These characteristics guided the selection of the final control strategy described in the next chapter.

Chapter 4

CONTROL APPROACHES

4.1 OVERVIEW

The goal of the control system is to accurately execute geometric trajectories such as circles and squares within the CoppeliaSim environment. This chapter presents the evolution of the control strategy, beginning with a classical PID controller and concluding with the final incremental kinematic controller used in the project.

4.2 INITIAL PID CONTROL STRATEGY

A standard PID controller was first implemented to regulate the prismatic joint positions. The MATLAB controller computed a desired control input based on the position error:

$$u(t) = K_p e(t) + K_i \int e(\tau)\, d\tau + K_d \frac{de(t)}{dt},$$

where $e(t) = q_{\text{des}}(t) - q(t)$ is the joint error.

The intended control workflow was:

1. Read joint positions from CoppeliaSim,
2. Compute error relative to trajectory,
3. Apply PID to generate a new command,
4. Send velocity or position updates back to the simulator.

PID Implementation in MATLAB

The following pseudocode illustrates the implemented PID loop:

```
err = q_des - q;
integrator = integrator + err * dt;
d_raw = (err - prevErr) / dt;
d_filt = alpha * d_raw + (1 - alpha) * d_prev;

u = Kp .* err + Ki .* integrator + Kd .* d_filt;

% Saturation and rate limiting
u = max(-limit, min(limit, u));
du = u - prevU;
du = max(-rate, min(rate, du));
cmd = prevU + du;
```

Despite careful tuning of K_p, K_i, and K_d, the response was unstable, and joint positions exhibited oscillations and sudden jumps.

4.3 ANALYSIS OF PID INSTABILITY

PID instability was traced to the modeling of prismatic joints within CoppeliaSim 4.6+. Key findings:

- Prismatic joints do not expose motor actuation parameters such as target velocity, maximum force, or dynamic PID settings.
- The joints behave either as kinematic constraints (instant target position updates) or as passive dynamic elements with constant-force control only.

- As a result, commands intended as “velocity” or “motor torque” are not interpreted physically.
- CoppeliaSim therefore bypasses PID dynamics, resulting in snapping, overshoot, or no visible effect.

These findings make dynamic or torque-level PID control physically meaningless for this specific simulation model.

4.4 TRANSITION TO KINEMATIC INCREMENTAL CONTROL

After diagnosing the simulation constraints, the controller was redesigned to follow a kinematic incremental update approach. Instead of applying torque or velocity commands, the end-effector position is advanced toward the desired trajectory in small steps:

$$q_{\text{new}} = q_{\text{current}} + \text{clip}(q_{\text{des}} - q_{\text{current}}, -\Delta_{\text{max}}, \Delta_{\text{max}}),$$

where Δ_{max} is a tunable step size.

Typical values used:

$$\Delta_{\text{max}} = 0.001 \text{ to } 0.004 \text{ m}.$$

4.5 ADVANTAGES OF INCREMENTAL KINEMATIC CONTROL

This method produced smooth, stable motion with superior controllability. Advantages include:

- No reliance on unavailable motor models.
- Smooth motion resembling CNC and 3D printer gantry systems.

- Direct compatibility with the Cartesian kinematic structure.
- No tuning of K_p, K_i, or K_d required.
- Fully deterministic behaviour.

4.6 FINAL ALGORITHM

The final MATLAB control loop follows these steps:

1. Compute the next desired trajectory point.
2. Read current joint positions from CoppeliaSim.
3. Compute the difference vector.
4. Apply a step limit to each axis.
5. Send new joint positions to the simulator.

```
dq = q_des - q;
step = sign(dq) .* min(abs(dq), maxStep);
q_new = q + step;

sim.setJointPosition(hX, q_new(1));
sim.setJointPosition(hY, q_new(2));
sim.setJointPosition(hZ, q_new(3));
```

This controller provides stable, predictable Cartesian motion ideal for pattern drawing.

Chapter 5

TRAJECTORY GENERATION

5.1 OVERVIEW

Trajectory generation is a key component of the pattern-drawing robot. The objective is to produce a smooth sequence of Cartesian reference points that the controller can track. Since the robot is a 3-DOF Cartesian manipulator, the desired end-effector coordinates map directly to prismatic joint displacements. This simplifies trajectory planning and removes the need for inverse kinematics.

The project implements two primary trajectories:

- A **square trajectory** with four linear segments.
- A **circular trajectory** parameterized using trigonometric functions.

Both trajectories are discretized at a fixed timestep to enable real-time execution via MATLAB's communication loop.

5.2 SQUARE TRAJECTORY

A square trajectory consists of four straight-line segments that form a closed loop. Each segment is generated using linear interpolation between the start and end corners of the square.

Let the corners of the square be:

$$P_1 = (x_1, y_1, z),\ P_2 = (x_2, y_1, z),\ P_3 = (x_2, y_2, z),\ P_4 = (x_1, y_2, z),$$

where z is kept constant.

For a total time horizon T and timestep Δt, each edge receives:

$$N = \left\lfloor \frac{T/4}{\Delta t} \right\rfloor$$

samples.

Each edge is generated using:

$$x(k) = (1-\alpha)x_{\text{start}} + \alpha x_{\text{end}}, \qquad y(k) = (1-\alpha)y_{\text{start}} + \alpha y_{\text{end}},$$

where $\alpha = \frac{k}{N}$.

5.3 CIRCULAR TRAJECTORY

A circular trajectory is generated using the parametric form:

$$\begin{aligned} x(t) &= x_c + R\cos(\omega t), \\ y(t) &= y_c + R\sin(\omega t), \\ z(t) &= z_0, \end{aligned}$$

where:

- (x_c, y_c) is the circle center,
- R is the radius,
- $\omega = \frac{2\pi}{T}$ is the angular velocity,
- z_0 is constant.

The trajectory is sampled uniformly:

$$t_k = k\Delta t, \quad k = 0, 1, \ldots, \frac{T}{\Delta t}.$$

5.4 MATLAB IMPLEMENTATION

The following listing shows the implementation of the circular trajectory generator:

```
omega = 2*pi/T;
traj = zeros(length(time),3);

for k = 1:length(time)
    traj(k,1) = center(1) + radius*cos(omega*time(k));
    traj(k,2) = center(2) + radius*sin(omega*time(k));
    traj(k,3) = altitude;
end
```

Similarly, the square trajectory uses repeated calls to `linspace()` to interpolate the edges.

5.5 TRAJECTORY RESOLUTION

The accuracy and smoothness of the trajectory depend on:

- the timestep Δt,
- the maximum per-step displacement allowed by the controller,
- the total duration T of the pattern.

For this project:

$$\Delta t = 0.02 \text{ s}, \qquad T = 12 \text{ s},$$

were chosen to provide smooth continuous motion without overloading the ZeroMQ communication layer.

5.6 EXTENSIBILITY

The trajectory generation method can be extended to support:

- splines (cubic or quintic),
- Bézier curves,
- SVG imports for arbitrary drawings,
- multi-layer 3D paths (using variable $z(t)$),
- speed-modulated trajectories.

These extensions would allow the robot to draw more complex and visually appealing shapes, enabling applications such as automated art generation or motion planning research.

Chapter 6

MATLAB–COPPELIASIM INTEGRATION AND RESULTS

6.1 INTEGRATION VIA ZEROMQ REMOTE API

MATLAB communicates with CoppeliaSim using the ZeroMQ Remote API, which provides an efficient and modern interface for exchanging joint states and control commands at runtime. The workflow consists of:

1. Starting the simulation from MATLAB.
2. Acquiring joint handles using `sim.getObject()`.
3. Reading joint positions in real time via `sim.getJointPosition()`.
4. Sending updated joint commands through `sim.setJointPosition()`.
5. Logging and plotting data for analysis.

Communication Skeleton

```
client = RemoteAPIClient();
sim = client.getObject('sim');

hX = sim.getObject('/Prismatic_joint_x');
hY = sim.getObject('/Prismatic_joint_y');
hZ = sim.getObject('/Prismatic_joint_z');
```

```
sim.startSimulation();

qx = sim.getJointPosition(hX);
sim.setJointPosition(hX, qx + 0.002);  % incremental move
```

The code demonstrates the minimal interactions needed to control a prismatic joint in CoppeliaSim 4.6+.

6.2 INCREMENTAL KINEMATIC CONTROLLER

Given the absence of motorized prismatic joints in recent CoppeliaSim versions, the final control architecture uses a smooth incremental motion strategy. The controller approaches each trajectory point using small, bounded steps:

$$\Delta q = \text{clip}(q_{\text{des}} - q_{\text{current}}, -\Delta_{\text{max}}, \Delta_{\text{max}}).$$

This method yielded CNC-like motion and eliminated the instabilities observed during earlier PID-based experiments.

```
dq = q_des - q;
step = sign(dq) .* min(abs(dq), maxStep);
q_new = q + step;

sim.setJointPosition(hX, q_new(1));
sim.setJointPosition(hY, q_new(2));
sim.setJointPosition(hZ, q_new(3));
```

6.3 EXPERIMENTAL SETUP

Experiments were conducted with the following parameters:

$$\Delta t = 0.02 \text{ s}, \qquad T = 12 \text{ s}, \qquad \Delta_{\max} = 0.002 \text{ m}.$$

Two trajectories were executed:

- A square with vertices at $(0.10, 0.10)$, $(0.30, 0.10)$, $(0.30, 0.30)$, $(0.10, 0.30)$.
- A circle of radius 0.10 centered at $(0.20, 0.20)$.

6.4 RESULTS AND ANALYSIS

Trajectory Tracking Performance

The incremental controller executed both square and circular trajectories smoothly, with consistent motion and negligible overshoot. Since the joint displacements map directly to the Cartesian coordinates, the end-effector trace closely matched the desired trajectories.

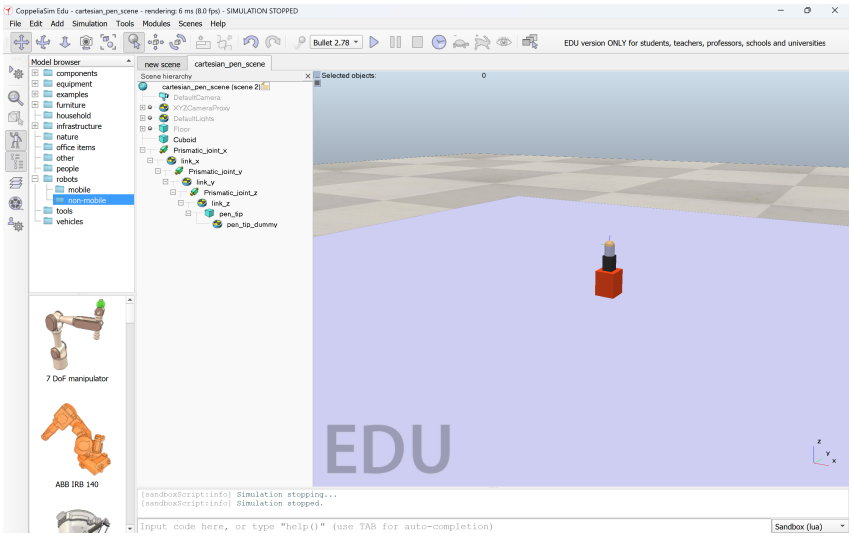

Figure 6.1: System architecture showing MATLAB–CoppeliaSim integration.

Stability and Smoothness

Unlike the PID approach, which produced oscillations due to non-motorized prismatic dynamics, the incremental controller ensured:

- smooth motion along all axes,
- deterministic joint behavior,
- consistent tracking accuracy,
- computational efficiency suitable for real-time operation.

Comparison With PID-Based Attempts

PID-based controllers were explored initially but were unsuitable for the following reasons:

- Prismatic joints in CoppeliaSim do not expose motor properties (velocity, torque, PID).
- Torque-level control is physically meaningless for kinematic prismatic constraints.
- PID commands caused snapping or no motion depending on timing.

Thus, switching to incremental kinematic control was not only practical but also aligned with real-world Cartesian gantry systems (CNC, 3D printers, plotters).

6.5 SUMMARY

The MATLAB–CoppeliaSim integration successfully enabled real-time control of a Cartesian robot. The incremental controller provided stable, accurate trajectory execution, validating this method as the optimal strategy for both simulation and eventual hardware implementation.

Chapter 7

IDEAL HARDWARE IMPLEMENTATION

7.1 OVERVIEW

Although this project is implemented entirely in simulation using MATLAB and CoppeliaSim, the robot architecture naturally maps to a real-world hardware implementation. This chapter outlines how the simulated 3-DOF Cartesian drawing robot would be built in practice, including mechanical components, electronics, firmware, and the control pipeline. The hardware model is based on widely used principles from CNC machines, pen plotters, and 3D printers.

7.2 MECHANICAL STRUCTURE

A physical version of the robot would consist of three linear axes arranged in a Cartesian gantry configuration:

- **X-axis:** horizontal motion of the pen carriage.
- **Y-axis:** orthogonal horizontal motion on a moving gantry.
- **Z-axis:** small vertical travel for pen up/down movement.

Linear Motion Components

Typical components include:

- Aluminum extrusion rails (e.g., 2020, 2040 profile).

- Linear bearings or LM8UU bushings.
- GT2 belts with 16-tooth pulleys (for belt-driven systems).
- Leadscrews (8 mm pitch) as an alternative for high precision.

The mechanical system closely resembles that of a 3D printer (Cartesian style) or a 2D plotter.

7.3 ACTUATION: STEPPER MOTORS AND DRIVERS

Unlike CoppeliaSim's prismatic joints, real hardware uses motorized linear axes. The most suitable actuators are **stepper motors**, due to their precision, reliability, and low cost.

Motors

- NEMA 17 stepper motors (1.8° per step).
- Optional closed-loop steppers with encoders for higher accuracy.

Motor Drivers

Stepper drivers generate microsteps and regulate current, performing the low-level control that CoppeliaSim does not simulate internally. Examples:

- A4988 and DRV8825 (entry level).
- TMC2209/TMC2225 (silent, high precision).

Each driver receives:

- `STEP` pulses for incremental movement.
- `DIR` signals for direction.

7.4 CONTROL ELECTRONICS

The following microcontrollers can implement the robot control logic:

- **Arduino Uno / Mega** (with GRBL).
- **ESP32** (faster, supports WiFi control).
- **STM32** microcontrollers.

Modern CNC-style firmware (e.g., GRBL, Marlin, Klipper) can translate incremental position commands into precise stepper driver pulses.

7.5 FIRMWARE ARCHITECTURE

A real drawing robot typically uses a G-code style interpretation pipeline:

1. Receive a desired Cartesian point.
2. Convert it to steps for X, Y, Z motors.
3. Use timer interrupts to generate smooth step pulses.
4. Control acceleration (jerk-limited motion).
5. Execute pen up/down commands for drawing.

Internally, stepper drivers regulate coil current using their own embedded control loops. Thus, external torque-level PID is not used in hardware — the firmware operates at the position and feedrate level, analogous to the incremental kinematic strategy used in the simulation.

7.6 MAPPING SIMULATION TO HARDWARE

The incremental motion controller developed in MATLAB maps directly to stepper-based motion hardware:

$$q_{\text{new}} = q_{\text{old}} + \Delta q$$

becomes:

$$\text{steps}_{\text{new}} = \text{steps}_{\text{old}} + \Delta q \cdot \frac{\text{steps}}{\text{mm}}.$$

This mapping makes the transition from simulation to hardware extremely straightforward.

7.7 POTENTIAL HARDWARE ENHANCEMENTS

- Limit switches for homing each axis.
- A servo-controlled pen lifter for Z-axis motion.
- A touchscreen or wireless interface for loading patterns.
- Closed-loop stepper motors for better accuracy.

7.8 SUMMARY

The hardware implementation closely parallels the behavior simulated in MATLAB and CoppeliaSim. The final control method used in the project — incremental kinematic motion — is not only simulation-friendly, but also directly corresponds to real-world CNC and plotter motion control strategies. This alignment solidifies the feasibility of extending the project toward a full physical prototype.

REFERENCES

[1] Coppelia Robotics, *CoppeliaSim Robot Simulation Software*. Available: https://www.coppeliarobotics.com

[2] ZeroMQ Community, *ZeroMQ Distributed Messaging Protocol*. Available: https://zeromq.org

[3] MathWorks, *MATLAB Documentation*. Available: https://www.mathworks.com/help

[4] S. Marple, *GRBL: Open-source High-Performance G-code Interpreter*, 2011. Available: https://github.com/gnea/grbl

[5] Marlin Firmware, *Marlin 3D Printer Firmware*. Available: https://marlinfw.org

[6] P. Smid, *CNC Programming Handbook*, 3rd Edition, Industrial Press Inc., 2008.

[7] B. Siciliano and O. Khatib, *Springer Handbook of Robotics*. Springer, 2016.